

Java programming.

```
import java.util.*; → packages
public class Main{
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

Access modifier ::

public, private, protected, default.

class ::

class is a blue print of an object. It has variables & method.

static ::

Without creating we can access a method.

void ::

It has no return type.

() → we can give arguments & parameters.

String [] args → It stores the logic in string array based.

{ } → 2 or more lines need to use {}.

; → If it complete in single line.

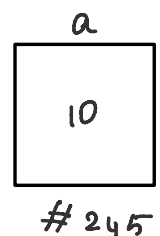
System.out.print("Hello"); → To print the output in a console.

println → next line.

Variable - It store the address of the value.

Rules of variable:

- * White space are not allowed.
- * Special characters are not allowed except (-, \$)
- * Number are not allowed in first.
- * Should not use pre-define name.



Data type

primitive (Not able to split)

Int, Byte, Short, long.

float, double

char, Boolean.

Non primitive. (Able to split)

Arrays, String, Class.

Class - user define
data type

Compile time initialization:-

```
import java.util.*;
public class Main{
    public static void main(String[] args) {
        int a = 10;
        double b = 22.23;
        System.out.println(a+" "+b);
    }
}
```

Run time initialization:-

```
import java.util.*;
public class Main{
    public static void main(String[] args) {
        //className objName = new ClassName();
        Scanner sc = new Scanner(System.in);
        float b = sc.nextFloat();
        int a = sc.nextInt();
        sc.nextLine();//enter space
        String str1 = sc.nextLine();// 2 or more words
        String str2 = sc.next();//single word
        char ch = sc.next().charAt(0);
    }
}
```

Object: Instance of a
class.

String:

H	e	l	l	o	o
0	1	2	3	4	5

operator

To perform a operation between two variable or operand.
operator are a Symbol.

Arithmetic operator:-

+, -, *, /, %.

Logical operator:-

And → &&, OR → ||, XOR → ^, Not → !

A	B	&&		^
1	1	1	1	0
1	0	0	1	1
0	1	0	1	1
0	0	0	0	0

1 - true
0 - false.

Relational operator:

<, <=, >, >=, ==, !=

Assignment operator: (=)

$a = a + 5 \rightarrow a += 5, a -= 5, a *= 5, a /= 5.$

unary operator: -

Increment / decrement

pre Increment $\rightarrow ++a$

post Increment $\rightarrow a++$

pre decrement $\rightarrow --a$

post decrement $\rightarrow a--$

```
public class Main{
    public static void main(String[] args) {
        int a = 5;
        int b = a++;
        int c = a++ + ++b;
        System.out.println(++c + ++a);
    }
}
```

Memory.

$a = 5 \rightarrow 7$

$b = 5$

$c = 6 + 6 = 12$

$13 + 8 = 21$

Ternary operator:

Advanced version of if condition.

Syntax:

$(\text{Condition}) ? \text{"true"} : \text{"false statement"};$

```
public class Main{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt();
        System.out.println((a>5)? "True": "False");
    }
}
```

Bitwise operator:

1 - True 0 - false.

Every values should be 0's & 1's.

$$n = 5 \rightarrow 101$$

Values are stored based on 2^n .

$$\begin{array}{cccccc} 2^4 & \dots & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\ & & \dots & 16 & 8 & 4 & 2 & 1 \\ & & & & & 1 & 0 & 1 \end{array} \rightarrow 5$$

$$\begin{array}{cccccc} 2^4 & \dots & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\ & & \dots & 16 & 8 & 4 & 2 & 1 \\ & & & 1 & 1 & 0 & 0 \end{array} \rightarrow 12$$

$$n = 8 \quad m = 12$$

$$8 \& 12 \Rightarrow$$

$$8 | 12 \Rightarrow$$

$$8 \wedge 12 \Rightarrow$$

$$\begin{array}{cccccc} 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\ 16 & 8 & 4 & 2 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ \hline 0 & 1 & 0 & 0 & 0 & \xrightarrow{\&} 8 \\ 0 & 1 & 1 & 0 & 0 & \xrightarrow{|} 12 \\ \hline 0 & 0 & 1 & 0 & 0 & \xrightarrow{\wedge} 4 \end{array}$$

Shift operator:

Left shift

$$5 \ll 2 \rightarrow \text{No 0's shift.}$$

$$\begin{array}{cccccc} 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\ 16 & 8 & 4 & 2 & 1 \end{array}$$

$$\begin{array}{cccccc} 0 & 0 & 1 & 0 & 1 \\ \swarrow & \swarrow & \swarrow & \swarrow & \swarrow \\ 0 & 0 & 1 & 0 & 1 \\ \swarrow & \swarrow & \swarrow & \swarrow & \swarrow \\ 0 & 0 & 1 & 0 & 1 \end{array} \quad \begin{array}{l} \textcircled{1} \\ \textcircled{2} \end{array}$$

$$16 + 4 = 20$$

$$5 \ll 2$$

left shift $\rightarrow \ll$
right shift $\rightarrow \gg$

right shift:

$$\begin{array}{cccccc} 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\ 16 & 8 & 4 & 2 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ \swarrow & \swarrow & \swarrow & \swarrow & \swarrow \\ 0 & 0 & 1 & 0 & 1 \end{array} \quad \textcircled{1}$$

$$5 \gg 1 = 2$$

HW:

Multiple by 2 without using arithmetic operator. $\ll 1$

Divided by 2 without using arithmetic operator. $\gg 1$

Conditional statement.

if, if-else, else-if, nested if, else if ladder, switch.

else-if ladder.

```
if (condition) {
    // statement;
}
else if (condition) {
    // statement;
}
else if (condition) {
    // statement;
}
else {
    // statement;
}
```

nested if:

```
if (condition) {
    if (condition) {
        // statement;
    }
    else {
        // statement;
    }
}
else {
    // statement;
}
```

Leap Year

4 years once.

1 year $\rightarrow$ 365 days 5 hr 48 min 48 sec.

1 year - 6h

2 year - 6h

3 year - 6h

4 year - 6h / 24h

8 year - 24h

96 year - 24h

100 year - 24h - 18h = 6h.

200 year - 24h - 18h = 6h

300 year - 24h - 18h = 6h

400 year - 24h - 18h = 6h / 24h

400, 800, 1200... $\rightarrow$ leap years.

12 min

6 hr.

100 $\rightarrow$ 18 h

year % 4 == 0

year % 100 != 0

year % 400 == 0

```
static boolean checkYear(int n) {
    if (n % 4 == 0) {
        if (n % 100 == 0) {
            if (n % 400 == 0) {
                return true;
            } else {
                return false;
            }
        } else {
            return true;
        }
    } else {
        return false;
    }
}
```

predict the output:-

```
public class Main{
    public static void main(String[] a) {
        int a = 5;
        int b = ++a;
        int c = b++ + a++;
        System.out.println(a++ + ++b - c-- + (++c + --a));
    }
}
```

Memory.

a = 5 6 7 8 7

b = 6 7 8

c = 6 + 6 = ~~12~~ 12

= 7 + 8 - 12 + (12 + 7)

= 7 + 8 - 12 + 19

= 7 + 8 + 7 = 22.

```
public class Main{
    public static void main(String[] args) {
        int a = 7;
        int b = --a;
        int c = ++a + --b + b--;
        int d = c--;
        System.out.println(--a + (a++ - c--)+ d++ - ++b);
    }
}
```

Memory.

a = 7 6 7 6 7

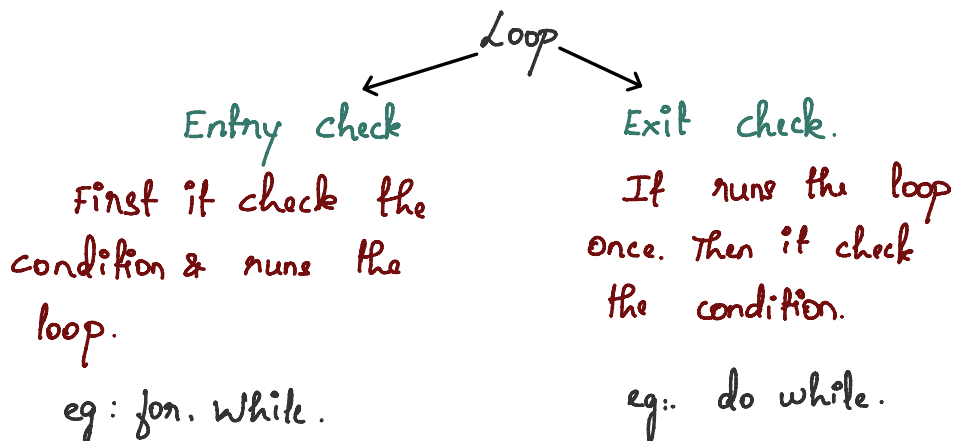
b = ~~6~~ 5

c = 7 + 5 + 5 = ~~17~~ 15

d = ~~17~~ 18

= 6 + (6 - 16) + 17 - 5

= 6 - 10 + 12 = 8//



For:-

Syntax:-

```
for (Initialization; condition; increment / decrement) {
    // logic
}
```

While:-

Syntax:-

```
Initialization;
while (condition) {
    // logic
    increment / decrement;
}
```

do while :-

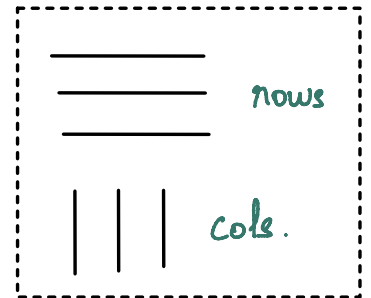
Syntax:-

```
Initialization;  
do {  
    // logic  
    increment / decrement;  
}  
while (condition);
```

Nested for loop:-

$n = 5$

```
for (int i = 0; i < n; i++) { → row  
    for (int j = 0; j < n; j++) { → col.  
        // logic  
    }  
}
```



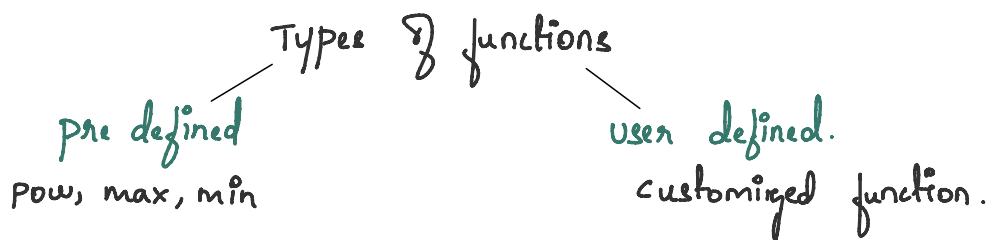
```
import java.util.*;  
public class Main {  
    public static void main(String[] args) {  
        for (int i = 0; i < 5; i++) { // 0 < 5  
            for (int j = 1; j <= 5; j++) {  
                System.out.print(i + "" + j + " ");  
            }  
            System.out.println();  
        }  
    }  
}
```

ij

01	02	03	04	05
11	12	13	14	15
21	22	23	24	25
31	32	33	34	35
41	42	43	44	45

Functions

It is a block of code that does a specific task.
We can reuse the functions.



User defined:-

- Without datatype without argument.
- Without datatype with argument.
- With datatype without argument.
- With datatype with argument.

Without returntype → void
 With returntype → Datatype.
 Without argument → ()
 With argument → (int a, int b)

- Function can be used using with static & without static.
- Function should be inside class & outside the method / function.

Without returntype without argument.

```
public class Main{
    public static void add(){
        int a = 4;
        int b = 5;
        System.out.println(a+b);
    }
    public static void main(String[] args) {
        add();
    }
}
```

```
import java.util.*;
public class Main{
    public void add(){
        int a = 4;
        int b = 5;
        System.out.println(a+b);
    }
    public static void main(String[] args) {
        Main obj = new Main();
        obj.add();
    }
}
```

Without returntype with argument.

```
import java.util.*;
public class Main{
    public static void add(int a, int b){
        System.out.println(a+b);
    }
    public static void main(String[] args) {
        add(5,6);
    }
}
```

```
import java.util.*;
public class Main{
    public void add(int a, int b){
        System.out.println(a+b);
    }
    public static void main(String[] args) {
        Main obj = new Main();
        obj.add(5,6);
    }
}
```

With returntype without argument.

```
import java.util.*;
public class Main{
    public static int add(){
        int a = 6;
        int b = 4;
        return a+b;
    }
    public static void main(String[] args) {
        int ans = add();
        System.out.println(ans);
    }
}
```

```
public class Main{
    public int add(){
        int a = 6;
        int b = 4;
        return a+b;
    }
    public static void main(String[] args) {
        Main obj = new Main();
        int ans = obj.add();
        System.out.println(ans);
    }
}
```

With return type with argument.

```
import java.util.*;
public class Main {
    public static int add(int a, int b) {
        return a+b;
    }
    public static void main(String[] args) {
        int ans = add(6,4);
        System.out.println(ans);
    }
}
```

```
import java.util.*;
public class Main{
    public int add(int a, int b){
        return a+b;
    }
    public static void main(String[] args) {
        Main obj = new Main();
        int ans = obj.add(6,4);
        System.out.println(ans);
    }
}
```

H/W:

Find odd or even without using arithmetic operator.

Square based Pattern problem.

Pattern 1:

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

```
import java.util.*;
public class Main {
    public static void Pattern(int n) {
        for(int i=0; i<n; i++) {
            for(int j=0; j<n; j++) {
                System.out.print("* ");
            }
            System.out.println();
        }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        Pattern(n);
    }
}
```

Pattern 2:

```
* * * * *
*       *
*       *
*       *
*       *
* * * * *
```

```
public static void Pattern(int n) {
    for(int i=0; i<n; i++) {
        for(int j=0; j<n; j++) {
            if(i==0 || j==0 || i==n-1 || j==n-1){
                System.out.print("* ");
            }
            else{
                System.out.print("  ");
            }
        }
        System.out.println();
    }
}
```

i\j	00	01	02	03	04
10	11	12	13	14	
20	21	22	23	24	
30	31	32	33	34	
40	41	42	43	44	

$i==0 || j==0 || i==n-1 || j==n-1$

Pattern 3:

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

```
public static void Pattern2(int n) {
    for(int i=0; i<n; i++) {
        for(int j=0; j<n; j++) {
            if(i==j || i+j == n-1) {
                System.out.print("* ");
            }
            else {
                System.out.print("  ");
            }
        }
        System.out.println();
    }
}
```

i,j

00	01	02	03	04
10	11	12	13	14
20	21	22	23	24
30	31	32	33	34
40	41	42	43	44

$i=j$ || $i+j=n-1$

Pattern 4:

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

```
public static void Pattern3(int n) {
    int mid = n/2;
    for(int i=0; i<n; i++) {
        for(int j=0; j<n; j++) {
            if(i==mid || j==mid) {
                System.out.print("* ");
            }
            else {
                System.out.print("  ");
            }
        }
        System.out.println();
    }
}
```

Pattern 5:

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

```
public static void Pattern5(int n) {
    int mid = n/2;
    for(int i=0; i<n; i++) {
        for(int j=0; j<n; j++) {
            if(j==0 || j==n-1 || i==j && i<=mid || i+j==n-1 && i<=mid) {
                System.out.print("* ");
            }
            else {
                System.out.print("  ");
            }
        }
        System.out.println();
    }
}
```

mid

00	01	02	03	04
10	11	12	13	14
20	21	22	23	24
30	31	32	33	34
40	41	42	43	44

$i=j$ && $i \leq mid$
 $i+j == n-1$ && $i \leq mid$

Pattern 6:-

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

```
public static void Pattern6(int n) {
    int mid = n/2;
    for(int i=0; i<n; i++) {
        for(int j=0; j<n; j++) {
            if(j==0 || j==n-1 || i==j && i>=mid || i+j==n-1 && i>=mid) {
                System.out.print("* ");
            }
            else {
                System.out.print("  ");
            }
        }
        System.out.println();
    }
}
```

Pattern 7:-

```

      <=  m  >=
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
      <=  m  >=

```

```

public static void Pattern7(int n) {
    int mid = n/2;
    for(int i=0; i<n; i++) {
        for(int j=0; j<n; j++) {
            if(i==mid || j==mid || i==0 && j<=mid ||
               i==n-1 && j>=mid || j==0 && i>=mid || j==n-1 && i<=mid) {
                System.out.print("* ");
            }
            else {
                System.out.print(" ");
            }
        }
        System.out.println();
    }
}

```

Pattern 8:-

```

* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *

```

```

00 01 02 03 04 05 06 07 08
10 11 12 13 14 15 16 17 18
20 21 22 23 24 25 26 27 28
30 31 32 33 34 35 36 37 38
40 41 42 43 44 45 46 47 48
50 51 52 53 54 55 56 57 58
60 61 62 63 64 65 66 67 68
70 71 72 73 74 75 76 77 78
80 81 82 83 84 85 86 87 88

```

$mid = 4$
 $4 + 8 = 12$
 $i + j = (n-1) + mid$

```

public static void Pattern8(int n) {
    int mid = n/2;
    for(int i=0; i<n; i++) {
        for(int j=0; j<n; j++) {
            if(i==0 && j<=mid || j==0 && i<=mid ||
               i==n-1 && j>=mid || j==n-1 && i>=mid || i==mid
               || i==j || i+j==mid || i+j==(n-1)+mid || j==mid) {
                System.out.print("* ");
            }
            else {
                System.out.print(" ");
            }
        }
        System.out.println();
    }
}

```

7/11/21

pattern 9:-

```

* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *

```

```

00 01 02 03 04
10 11 12 13 14
20 21 22 23 24
30 31 32 33 34
40 41 42 43 44

```

$i - j == mid$ || $i + j == (n-1) + mid$